

# Post-WhatsApp Rollback Workflow Refinement Prompt - 2026-08-01

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, QA lead, and SaaS operations advisor.

Inspect Stoquify thoroughly after the WhatsApp integration rollback. Identify every workflow, surface, service, action, component, test, setting, report, and user journey that was affected by either the original WhatsApp integration or the rollback. Then remodel and refactor only the affected workflows so they become simpler, more professional, modern, enterprise-grade, secure, transparent, and easier to maintain.

## Goal

Improve the restored non-WhatsApp workflows so they feel smooth, reliable, secure, and operationally clear without reintroducing WhatsApp or replacing it with another messaging provider.

## Primary Focus

- POS receipt workflow
- POS sale commit flow
- Receipt delivery service and schemas
- Notification/settings surfaces
- Communication-related abstractions left behind by the rollback
- Tests affected by receipt/notification changes
- User-facing copy and empty/error/success states
- Audit, RBAC, tenant isolation, and module entitlement behavior around affected workflows

## Execution Rules

- Do not reintroduce WhatsApp runtime behavior, WhatsApp UI, WhatsApp environment variables, or WhatsApp provider abstractions.
- Do not replace WhatsApp with a new provider unless explicitly requested later.
- Do not weaken RBAC, tenant isolation, audit logging, policy gates, module entitlement, or security boundaries.
- Do not perform broad rewrites or unrelated cleanup.
- Do not use destructive git commands.
- Preserve unrelated user changes in the dirty worktree.
- Prefer small, durable refactors that reduce complexity and improve clarity.
- If a workflow's intended baseline is unclear, stop and produce a blocker report instead of guessing.

## Inspection Checklist

- 1 Read the rollback evidence report under `what - next /`.
- 2 Inspect all files changed by the WhatsApp rollback.

- 3 Search for remaining receipt, notification, communication, and provider abstractions that became unnecessary or unclear after rollback.
- 4 Map the affected workflows end to end:
  - POS sale creation
  - Receipt channel selection
  - Receipt delivery behavior
  - Receipt failure handling
  - Notification preferences
  - UI success/error states
  - Tests and policy gates
- 1 Identify complexity, duplication, unclear ownership, weak errors, confusing UI states, and security risks.

## Implementation Checklist

- 1 Simplify receipt schemas and service boundaries where safe.
- 2 Make receipt delivery behavior explicit, predictable, and easy to test.
- 3 Improve UI state clarity for receipt channel selection and delivery outcomes.
- 4 Ensure error messages are professional, safe, and useful.
- 5 Remove dead abstractions left behind by the rollback.
- 6 Add or update focused tests for every touched workflow.
- 7 Keep all changes narrow and traceable to the rollback-affected surfaces.
- 8 Save an evidence report under:

```
what-next/POST_WHATSAPP_ROLLBACK_WORKFLOW_REFINEMENT_REPORT_YYYY-MM-DD.md
```

## Verification

- Run focused POS, receipt, and notification tests first.
- Run broader relevant tests only if shared services or policy-sensitive surfaces are touched.
- If relevant, run `npm run policy:gates`.
- Record all commands and results in the evidence report.

## Expected Final Report

Include:

- Workflows inspected.
- Problems found after rollback.
- Refactors implemented.
- Files changed.
- Tests added or updated.
- Commands run and results.
- Remaining risks or blockers.

- Confirmation that WhatsApp was not reintroduced.
- Confirmation that unrelated workflows were not intentionally changed.

## Success Criteria

- Restored non-WhatsApp workflows are simpler and clearer.
- POS receipt behavior is predictable, test-covered, and provider-neutral.
- UI states are calm, professional, and understandable.
- Error handling is safe and enterprise-grade.
- No active WhatsApp runtime, UI, config, or provider integration remains.
- Focused tests pass.
- Evidence report is saved under `what-next/`.